

Tytuł projektu: 25 WYKORZYSTANIE DUŻYCH MODELI JĘZYKOWYCH (LLM) DO
AUTOMATYCZNEGO
GENEROWANIA I ANALIZY GRAFÓW INFRASTRUKTURY LINIOWEJ Z UŻYCIEM BAZ
GRAFOWYCH

Autorzy: Mateusz Młynek, Michał Szczepanik.

Przedmiot: Hurtownie Danych i Systemy Eksploracji Danych

Grupa: Grupa nr. 1 (usos), BDilS-1

Data: 25.05.2026 r.

Cel projektu:

Stworzenie prototypu systemu, który konwertuje nieustrukturyzowane opisy tekstowe na ustrukturyzowany format grafowy. Projekt miał na celu ocenę zdolności modeli LLM do identyfikacji obiektów technicznych (np. mostów, skrzyżowań) oraz implementację bazy Neo4j pozwalającej na zaawansowaną analizę topologiczną.

Rozwiązywany problem: Tradycyjne metody modelowania infrastruktury liniowej opierają się na ręcznej digitalizacji. Proces ten jest bardzo czasochłonny, wysoce kosztowny oraz obciążony dużym ryzykiem błędów ludzkich.

Zakres wykonanych prac:

- Przygotowanie listy obiektów, relacji oraz atrybutów które będzie musiał definiować model
- Przygotowanie złotego standardu z realnych tekstów z którymi będzie porównywany model
- Opracowanie architektury potoku danych (Data Pipeline).
- Przygotowanie inżynierii promptów, w tym rygorystycznych instrukcji systemowych (System Prompt) i wzorców metodą Few-Shot Learning, aby zmusić model do zwracania czystego kodu JSON.
- Wykorzystanie API modelu Gemini do wyodrębniania encji i relacji z surowych tekstów.
- Implementacja skryptu ładującego JSON do bazy Neo4j przy wykorzystaniu języka zapytań Cypher.
- Przeprowadzenie eksperymentów analitycznych, w tym ewaluacji metryk (Recall, Precision, F1-Score) oraz badanie wpływu temperatury modelu na jakość i stabilność wyników (zjawisko fuzji wiedzy i tzw. "wysp danych").

Dane wejściowe

Pochodzenie: Surowe, nieustrukturyzowane dane tekstowe pochodziły z publicznych komunikatów prasowych i aktualizacji technicznych publikowanych przez Generalną Dyрекcję Dróg Krajowych i Autostrad (GDDKiA).

Format: Dokumenty tekstowe (tekst naturalny).

Zawartość: Opisy planowanych modernizacji, budowy obwodnic, a także analizy infrastrukturalne dróg (m.in. rozbudowa autostrady A4, węzeł Krzyszkowice na DK7. Teksty zawierały parametry inżynieryjne dróg, obiektów i planowanych relacji przestrzennych.

Teksty na których były przeprowadzane próby:

<https://www.gov.pl/web/gddkia/kompleksowe-zmiany-na-a4>

<https://www.gov.pl/web/gddkia-krakow/wezel-krajowy-drogi-lokalne>

Założenia projektowe:

-Na początku prób model był testowany zarówno w trybie stochastyczny jak i deterministycznym, w dalszej części prób przyjęto w pełni deterministyczny tryb pracy modelu LLM (temperatura $T=0.0$) – było to uznane za warunek konieczny z punktu widzenia inżynierskiego dla zachowania stabilności strukturalnej plików JSON oraz zapewnienia powtarzalności wyników.

-Założono wykorzystanie konkretnej, sztywnej ontologii (słownika), definiującej dozwolone węzły (Odcinek drogi, Skrzyżowanie, Rondo, Obiekt inżynierski, Punkt POI) oraz konkretne relacje pomiędzy nimi (np. ŁĄCZY_SIE_Z, ZAWIERA).

-Pominięto atrybuty, o których brakowało informacji w tekście źródłowym.

-Integracja z bazą grafową neo4j

Opis rozwiązania

Działanie systemu: System pobiera surowy komunikat GDDKiA, a następnie model językowy (Gemini 2.5 Flash), zasilony promptem systemowym, dokonuje analizy semantycznej. Wyciągnięte informacje są formatowane przez LLM wyłącznie do struktury JSON. Następnie specjalny mechanizm importuje wygenerowany JSON do chmurowej bazy Neo4j.

Architektura i narzędzia: Całość logiki biznesowej oparto na języku Python 3.10+ (integrując biblioteki google-genai oraz neo4j Python Driver). Do przesyłania poleceń do bazy zastosowano zapytania Cypher

Model danych / baza danych (Grafowa)

Struktura została sprowadzona do grafu skierowanego:

Węzły:

- Odcinek drogi (atrybuty: nawierzchnia, długość, liczba pasów, klasa drogi).
- Skrzyżowanie (atrybuty: rodzaj, sygnalizacja świetlna).
- Rondo (atrybuty: liczba wlotów, nazwa, lokalizacja).
- Obiekt inżynierski (typ: Most/Wiadukt/Tunel, nośność, opis).
- Punkt POI (kategoria np. MOP/Stacja, nazwa własna).

Relacje:

- ŁĄCZY_SIE_Z: Pomiędzy Odcinkiem a Skrzyżowaniem/Rondem.
- ZAWIERA: Odcinek zawiera Obiekt inżynierski lub rondo.
- PRZECHODZI_NAD/POD: Obiekt inżynierski <-> droga / przeszkoda.
- DOJAZD_DO / PRZY_DRODZE: Połączenie odcinka z usługami POI.

Implementacja/walidacja

Kod: Oparto się o skrypt w języku Python korzystający z parametrów `response_mime_type="application/json"`, by narzucić format zwracanych przez LLM danych. Użyto pętli do iteracji przez właściwości JSON-a i przetłumaczenia ich na komendy Cypher.

Biblioteki standardowe (wbudowane w Pythona):

- **json**: Służy do parsowania tekstowych odpowiedzi z Gemini na słowniki Pythona, formatowania danych oraz zapisu wyników do pliku `.json`.
- **os**: Odpowiada za operacje na systemie plików – tworzenie folderów, sprawdzanie czy pliki istnieją oraz listowanie zawartości katalogu.

Biblioteki zewnętrzne (wymagające instalacji):

- **google-genai** (`from google import genai`): Oficjalny, nowoczesny SDK od Google do komunikacji z modelami Gemini. Wykorzystuje parametr `response_mime_type="application/json"`, aby zagwarantować stabilną strukturę wyjściową.
- **neo4j**: Oficjalny sterownik (*driver*) do bazy danych Neo4j. Odpowiada za nawiązanie bezpiecznego połączenia i wykonywanie zapytań w języku **Cypher**

Walidacja poprawności: Program przetestowano metodą wielokrotnych, empirycznych prób przy różnej temperaturze modelu. Testy wykazały, że przy $T > 0.0$ pojawiał się "szum tokenizacji" (np. błędy w nazwie relacji jak ŁĄCZY_SIŁĘ_Z), który powodował wywrotki bazy danych. Wyciągnięto wniosek, że LLM nie jest systemem w 100% samowystarczającym.

Analiza wyników

Metryki skuteczności: Wyniki pokazały rewelacyjną Czułość (Recall) wynoszącą 100% – model nie pominął kluczowych obiektów infrastruktury ujętych w tekstach. Precyzja (Precision) oscylowała w okolicach 77%, gdyż LLM często realizował tzw. "nad-ekstrakcję" (rozszerzał graf o poprawne logicznie węzły satelickie/kontekstowe, których nie było w ziarnie). Wynik F1-Score to 87.2%, co uznano za bardzo dobry.

Zjawiska grafowe: Model wykazał zdolność zwaną "Entity Resolution". Przeanalizowano połączone teksty z dwóch dokumentów (A4 oraz DK7). Podczas "Fuzji Wiedzy", AI samoistnie połączyło oba teksty w jedną topologię używając miasta (Kraków) jako węzła kotwiczącego.

Odnotowano jednak również "awarie", gdzie model tworzył dwie niepołączone "wyspy danych", traktując wspólną miejscowość jako zwykły tekst w opisach.

Poniżej zdjęcia (grafy na zdjęciach pochodzą z bazy neo4j)

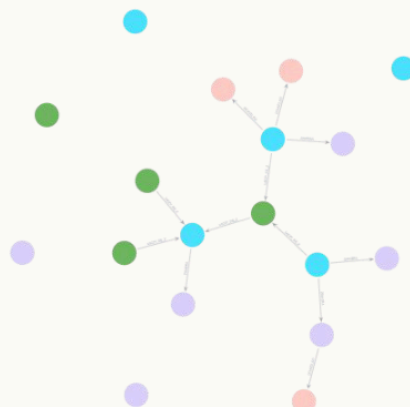
Ziarno

Case Study: Modernizacja Zakopianki (DK7)

Analiza tekstu technicznego dotyczącego przebudowy układu komunikacyjnego w Krzyszkowicach. Kluczowym wyzwaniem była ekstrakcja relacji między trasą główną (tranzytową) a nowymi drogami lokalnymi.

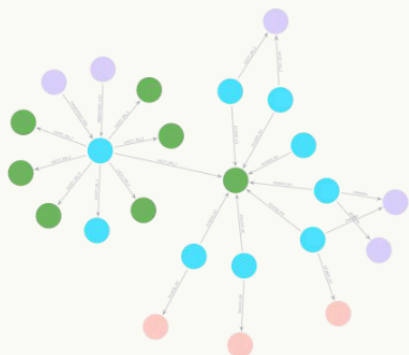
Specyfika Danych:

- ✓ **Obiekty:** Tunel pod DK7, mosty na potokach Krzyszkowianka i Młynówka.
- ✓ **Relacje:** Likwidacja lewoskrętów na rzecz bezkolizyjnego węzła.
- ✓ **Punkty POI:** Przysiółki (Bugaj, Psiary) jako punkty docelowe dróg lokalnych.



Wynik:

Wizualizacja Wyników w Neo4j



Struktura wygenerowanego grafu

Na powyższym schemacie widoczna jest bezpośrednia implementacja wyekstrahowanych danych w bazie Neo4j:

-  **Węzły centralne:** Reprezentują kluczowe odcinki dróg (np. A4, DK7).
 -  **Obiekty inżynieryjne:** Obiekty inżynieryjne (Wiadukty, Tunele) osadzone na trasie poprzez relację ZAWIERA.
 -  **Skrzyżowania**
 -  **POI**
- **Topologia:** Strzałki relacji precyzyjnie odwzorowują kierunki potoków ruchu opisane w komunikatach prasowych.

Ekstrakcja Wiedzy: Ziarno vs Próba nr 1

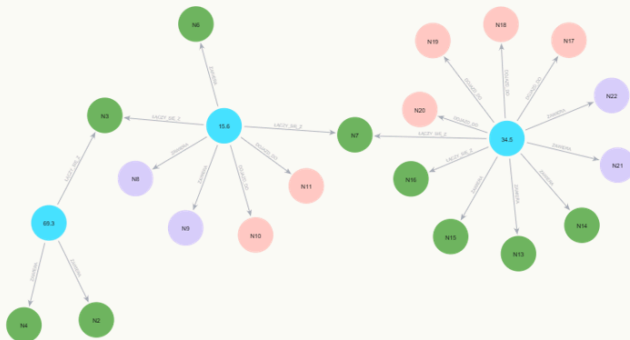
Metryka / Element	Wzorzec (Ziarno)	Wynik LLM (Próba nr 1)	Wniosek
Liczba Węzłów	18 węzłów	24 węzły	Nad-ekstrakcja: model wyodrębnił dodatkowe skrzyżowania (Mogilany, Głogoczów).
Klasyfikacja Dróg	"GP" dla DK7	"DK" oraz "lokalna"	Model wprowadził semantyczny podział klas dróg, którego nie było w ziarnie.
Typy Relacji	ŁĄCZY_SIĘ_Z	DOJAZD_DO	Różnica logiczna: LLM zinterpretował połączenie drogi z węzłem jako funkcję dojazdu.
Relacje Przestrzenne	ZAWIERA (Obiekty)	PRZECHODZI_NAD	Sukces: Model poprawnie zidentyfikował bezkolizyjność kładek dla pieszych.

Obserwacja kluczowa: Próba nr 1 wykazuje tendencję do "wzbogacania" grafu o dodatkowe punkty węzłowe wspomniane w tekście pobocznie (tzw. węzły satelickie), co zwiększa ziarnistość danych kosztem precyzji względem surowego wzorca.

Ziarno:

Case Study 2: Rozbudowa Autostrady A4

Analiza objęła blisko 120-kilometrowy odcinek autostrady A4 w województwach śląskim i małopolskim. Był to najbardziej wymagający tekst pod kątem gęstości informacji technicznych.



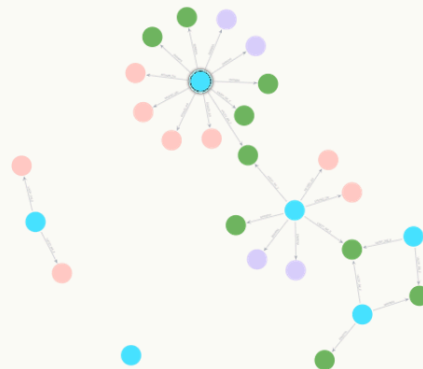
Wynik:

Case Study: Rozbudowa Autostrady A4

Złożoność Ekstrakcji Wiedzy

Eksperyment na danych A4 wykazał najwyższą "inteligencję topologiczną" modelu LLM. System nie tylko przeniósł dane, ale zaproponował własną strukturę logiczną.

- Odkrycie Hierarchii:** Model zidentyfikował węzeł nadrzędny "Autostrada A4" (669 km), spinający wszystkie pod-odcinki.
- Kontekst Miejski:** Samodzielna ekstrakcja miast (Katowice, Kraków) jako punktów docelowych/węzłowych.
- Relacje Inkluzji:** Poprawne zastosowanie relacji ZAWIERA dla MOP-ów i mostów.



Analiza Porównawcza: Ziarno vs Próba (A4)

Cecha Strukturalna	Wzorzec (Ziarno)	Wynik LLM (Próba nr 1)	Komentarz Badawczy
Liczba Węzłów	22 węzły	27 węzłów	Zwiększona ziarnistość przez dodanie miast i węzła głównego.
Topologia	Płaska (lista odcinków)	Hierarchiczna (Rodzic-Dziecko)	Model LLM poprawnie zdefiniował A4 jako "kontener" dla mniejszych dróg.
Kompletność (Recall)	100%	Zero pominieć (False Negatives) w kluczowych mostach i MOP-ach.	
Relacje POI	DOJAZD_DO	ZAWIERA / DOJAZD_DO	Model użył relacji zawierania dla MOP-ów, co jest technicznie zasadne.

Wniosek: Różnica między Ziarnem a Próba wynika z **pozytywnej nad-ekstrakcji**. Model LLM "rozumie" infrastrukturę lepiej niż surowy algorytm, uzupełniając brakujące powiązania logiczne między regionami a drogą.

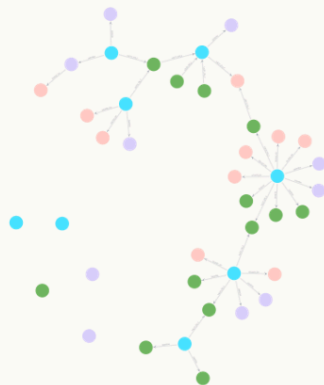
Ziarno:

Fuzja Wiedzy: Eksperyment Zintegrowany (A4 + DK7)

Konceptcja „Super-Ziarna”

Przetworzenie połączonych komunikatów technicznych w ramach jednego okna kontekstowego (Single-Context Window) w celu uzyskania spójnej sieci regionalnej.

- Rezolucja Identyfikatorów:** Przesunięcie sekwencyjne ID (A4: N1-N22, Zakopianka: N24-N41) w celu uniknięcia kolizji w bazie danych.
- Podjęcie Single-Context:** Pozwala modelowi LLM na jednoczesny dostęp do obu tekstów, co jest kluczem do odnalezienia punktów wspólnych.



Wynik

Próba nr 1: Sukces Rezolucji Encji i Fuzji – Kraków jako „Anchor Node”

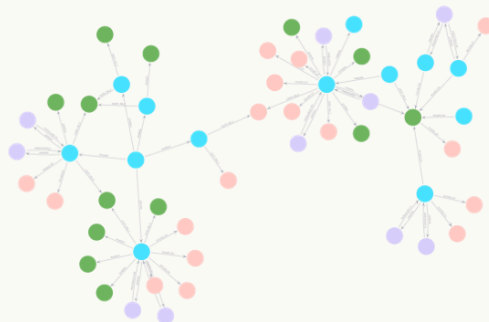
Prawidłowa Abstrakcja Geograficzna

Model w tej próbie zachował się jak rasowy inżynier GIS – dostrzegł, że „Kraków” to punkt centralny dla obu systemów drogowych.

- Utworzenie Węzła Centralnego:** Model wyodrębnił węzeł N26 (Miasto: Kraków).
- Kotwiczenie Autostrady A4:** Odcinek Katowice–Kraków (N24) został połączony bezpośrednią relacją ŁĄCZY_SIE_Z z Krakowem (N26).
- Kotwiczenie Zakopianki:** Odcinek DK7 Kraków–Myślenice (N40) również został połączony relacją ŁĄCZY_SIE_Z z tym samym Krakowem (N26).

Efekt w bazie Neo4j:

Dzięki spięciu obu dróg przez wspólny punkt referencyjny (Węzeł N26), algorytmy przeszukiwania grafu (np. Dijkstra/A*) mogą bez przeszkód wyznaczać trasy tranzytowe z Katowic prosto w Tatry. Fuzja zakończyła się pełnym sukcesem topologicznym.



Wynik 2

Próba nr 2: Awaria Fuzji ("Wyspy Danych")

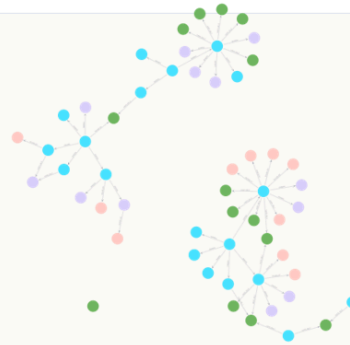
Błąd Semantycznego Rozproszenia

W tej próbie model stracił zdolność abstrakcji encji nadrzędnych. Zamiast stworzyć jeden węzeł dla miasta Kraków, zamknął informację geograficzną wewnątrz stringów.

- ✖ **Brak Węzła Wspólnego:** W spisie 55 węzłów nie istnieje dedykowany obiekt dla Krakowa ani Balic (Balice pojawiają się tylko jako Punkt POI N28, ale nie są połączone z DK7).
- ⚠ **Informacja jako Tekst, nie Obiekt:** Słowo "Kraków" zostało uwiecznione w nazwach odcinków: N25 ("A4 Katowice - Kraków"), N27 ("A4 Katowice - Balice"), N32 ("Zakopanka Kraków - Myślenice").

Skutek dla bazy danych:

Analiza relacji w Próbie nr 2 wykazuje ****całkowity brak połączenia**** pomiędzy podsiecią A4 a podsiecią DK7. Powstały dwie całkowicie odizolowane "wyspy danych". System komputerowy analizujący ten graf uznałby, że z autostrady A4 nie da się zjechać na Zakopankę, co eliminuje ten graf z użytku inżynierskiego.

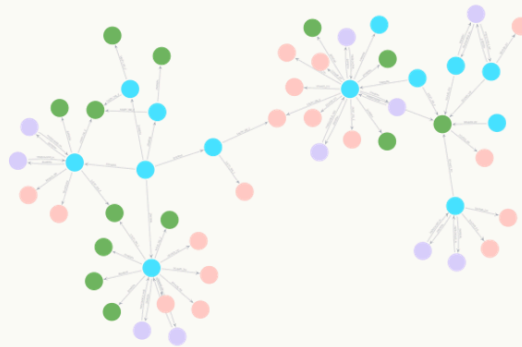


Fuzja Wiedzy i Entity Resolution

Integracja Wieloźródłowa

Największym sukcesem projektu była zdolność modelu do autonomicznego łączenia grafów z różnych dokumentów (A4 oraz DK7).

- 🔗 **Węzeł Kotwiczący:** Automatyczne rozpoznanie "Krakowa" jako wspólnego punktu referencyjnego.
- 🔗 **Scalenie Topologii:** Połączenie końca Autostrady A4 z początkiem Zakopianki w jedną sieć.
- 🔗 **Rezultat:** Przejście z odizolowanych "wysp danych" do zintegrowanego grafu transportowego regionu.



Ewaluacja Fuzji Wiedzy: Ziarno vs Próba 1 i 2

Cecha Strukturalna	Wzorzec (Ziarno Małopolski)	Próba nr 1 (Sukces Fuzji)	Próba nr 2 (Anomalia Strukturalna)
Liczba Węzłów	41 węzłów	53 węzły (Nad-ekstrakcja kontekstowa)	55 węzłów (Nadmierna segmentacja)
Encja Kotwicząca (Kraków)	Wyodrębniona jako osobny węzeł POI (N23)	Wyodrębniona jako osobny węzeł Miasto (N26)	Brak węzła. Rozproszona w nazwach właściwości odcinków.
Topologia Sieci	Zintegrowany graf (Jedna sieć)	Połączona spójnie	Rozbita (2 odizolowane subgrafy)
Relacje Nadrzędne	Płaskie mapowanie ciągów	A4 zdefiniowana jako kontener makro ("N1")	A4 zdefiniowana jako kontener makro ("N1")

Wniosek metodologiczny: Mimo identycznego promptu i reżimu temperaturowego ($T=0.0$), wysokie zagęszczenie informacji w tekście zintegrowanym wywołało fluktuację probabilistyczną, prowadząc do dwóch skrajnie różnych strategii mapowania świata przez LLM.

Analiza Anomalii: Tokenization Noise

Zjawisko błędu syntaktycznego

Eksperymenty w trybie stochastycznym ujawniły błędy tokenizacji, które uniemożliwiają automatyczny import do bazy Neo4j.

```
"source": "N3",
"target": "N9",
"type": "ŁĄCZY SIĘ Z"
```

Błąd: Nadmiarowy token "I" w nazwie relacji.

Rozwiązanie: Wdrożenie warstwy walidacji **Pydantic** wymuszającej zgodność ze schematem przed importem.

Metryki Jakościowe i Ilościowe

100%

Recall (Pełność)

~77%

Precision (Precyzja)

87.2%

F1-Score

0.75

Indeks Jaccarda

Interpretacja wyników: Stuprocentowa pełność (Recall) gwarantuje, że model nie pomija kluczowych danych technicznych. Niższa precyzja wynika z "nad-ekstrakcji" (tworzenia dodatkowych węzłów kontekstowych dla miast i regionów), co w kontekście budowania bazy wiedzy jest traktowane jako cecha pozytywna (Knowledge Enrichment).

Podsumowanie / Źródła

Osiągnięcia: Osiągnięto gigantyczną przewagę kosztową i czasową, skracając analizę danych z godzin do sekund. Wskazano, że graf Neo4j jest naturalnie lepszy niż zwykłe relacyjne bazy tabelaryczne do mapowania sieci topologicznej.

Rekomendacje na przyszłość (Dalsze prace): Jako optymalne wdrożenie zalecono wykorzystanie zasady "Human-in-the-Loop", by wygenerowany przez maszynę graf-szkic sprawdzał inżynier. Rekomenduje się również migrację z płatnych API (Gemini/OpenAI) na otwarte modele instalowane lokalnie (np. Llama-3, Mistral), co zapewni darmowe operacje i chroni wrażliwe dane przetargowe GDDKiA.

Bibliografia / Źródła: Bezpośrednie adresy do artykułów / dokumentacji prasowych GDDKiA w domenie gov.pl. **Technologie:** Python, Neo4j Aura, Cypher, Google Gemini Flash. Teksty na których były przeprowadzane próby:

<https://www.gov.pl/web/gddkia/kompleksowe-zmiany-na-a4>

<https://www.gov.pl/web/gddkia-krakow/wezel-krajowy-drogi-lokalne>

Plusy (Silne Strony) Zaproponowanej Metody

Efektywność i Skalowalność

- ⚡ Radykalna automatyzacja:** Skrócenie czasu ekstrakcji danych z dokumentacji technicznej z godzin (praca manualna) do sekund.
- 📁 Przetwarzanie masowe:** Możliwość równoległej analizy setek komunikatów i raportów bez wzrostu zapotrzebowania na zasoby ludzkie.

Głęboka Semantyczna i Fuzja

- 🔍 Recall na poziomie 100%:** Gwarancja, że żaden kluczowy obiekt infrastruktury wymieniony w tekście nie zostanie pominięty.
- 🧩 Autonomiczne Entity Resolution:** Zdolność do samodzielnego łączenia niezależnych dokumentów w jedną logiczną sieć grafową (np. wokół węzłów miejskich).

Minusy (Ograniczenia i Słabe Strony)

Problemy Generatywne i Składniowe

- ⚠ Szum Tokenizacji ($T > 0.0$):** Ryzyko powstawania błędów w nazwach relacji (np. "ŁĄCZY_SIE_Z"), blokujących import do bazy.
- 🔴 Nad-ekstrakcja (False Positives):** Tworzenie nadmiarowych, ogólnych węzłów kontekstowych (np. regionów), co obniża precyzję.

Zależności Technologiczne

- 🔧 Wrażliwość na Prompting:** Wynik końcowy jest silnie uzależniony od precyzyjnej konstrukcji instrukcji systemowych i przykładów Few-Shot.
- 💰 Koszty utrzymania:** Komercyjne API (OpenAI/Gemini) generują koszty proporcjonalne do wolumenu tokenów przy skomplikowanych promptach.

Kluczowe Wnioski Badawcze i Metodologiczne

- 🔗 Prymat Pełnego Determinizmu (T=0.0):** Dla systemów zasilających relacyjne lub grafowe bazy danych, temperatura zero jest warunkiem koniecznym do zachowania spójności schematu.
- 🛡️ Niezbędność Walidacji Hybrydowej:** Sam model LLM nie jest systemem autonomicznym. Konieczne jest stosowanie zewnętrznych parserów (np. Pydantic, JSON Schema) jako "bezpieczników" strukturalnych.
- 🗺️ Graf jako Optymalny Model Świata:** Baza Neo4j idealnie odzwierciedla nieliniową i wielopoziomową naturę opisów infrastruktury liniowej, przewyższając klasyczne tabele relacyjne.

Rekomendacje dla Wdrożeń Przemysłowych

Architektura Systemu

Podjęcie Human-in-the-Loop: System powinien generować graf jako "wersję kandydacką" (wstępną), która podlega szybkiej, wizualnej akceptacji przez inżyniera kontraktu, eliminując błędy typu False Positive.

Optymalizacja i Bezpieczeństwo

Lokalne Modele LLM (Open-Source): W celu redukcji kosztów API oraz ochrony danych wrażliwych (np. niepubliczne plany przetargowe), rekomenduje się migrację na wyspecjalizowane, lokalne modele klasy Llama-3 lub Mistral dostrojone do języka polskiego.

Kod:

```
import json
import os
from google import genai
from google.genai import types
from neo4j import GraphDatabase

# --- KONFIGURACJA ---
NEO4J_URI =
NEO4J_USER =
NEO4J_PASSWORD =
GEMINI_API_KEY =
# Inicjalizacja nowego klienta Gemini
```

```
client = genai.Client(api_key=GEMINI_API_KEY)
```

```
# --- SYSTEM PROMPT ---
```

```
SYSTEM_PROMPT = ""
```

Jesteś ekspertem w dziedzinie inżynierii drogowej i analizy danych grafowych. Twoim zadaniem

jest przekształcanie tekstowych opisów infrastruktury liniowej na ustrukturyzowany format JSON, który zostanie zaimportowany do bazy Neo4j.

1. DOPUSZCZALNE WĘZŁY I ATRYBUTY:

- Odcinek drogi: {nazwa, nawierzchnia, dlugosc, liczba_pasow, klasa}
- Skrzyżowanie: {nazwa, rodzaj, sygnalizacja (Tak/Nie)}
- Rondo: {nazwa, liczba_wlotow, lokalizacja}
- Obiekt inżynierski: {typ_obiektu (Most/Wiadukt/Tunel/Przepust), nazwa, nosnosc}
- Punkt POI: {kategoria, nazwa_wlasna}

2. DOPUSZCZALNE RELACJE:

- ŁĄCZY_SIE_Z: (Odcinek <-> Skrzyżowanie/Rondo)
- ZAWIERA: (Odcinek -> Obiekt inżynierski/Rondo)
- PRZECHODZI_NAD / PRZECHODZI_POD: (Obiekt inżynierski <-> Droga)
- DOJAZD_DO: (Odcinek/Obiekt -> Punkt POI)

3. ZASADY EKSTRAKCJI:

- Zwracaj wyniki WYŁĄCZNIE w formacie JSON.
- Każdy węzeł musi mieć unikalne ID (np. N1, N2).
- Jeśli brakuje jakiegoś atrybutu w tekście, pomiń go w obiekcie properties.

4. PRZYKŁADY (Few-Shot):

```
{
  "document_info": {
    "title": "Obwodnica Przecławia i Warzymic (DK13)",
    "date": "2023-08-31",
    "source_type": "Real-world technical description"
  },
  "nodes": [
    {
      "id": "N1",
      "type": "Odcinek drogi",
      "properties": {
        "nazwa": "Obwodnica Przecławia i Warzymic - Etap I",
        "dlugosc": 4.2,
        "liczba_pasow": 4,
        "klasa": "GP"
      }
    },
    {
      "id": "N2",
      "type": "Odcinek drogi",
      "properties": {
        "nazwa": "Obwodnica Przecławia i Warzymic - Etap II",
        "dlugosc": 2.3,
        "klasa": "S/GP"
      }
    },
    {
      "id": "N3",
      "type": "Odcinek drogi",
      "properties": {
```

```
"nazwa": "Obwodnica Kołbaskowa",
"długość": 5.6,
"liczba_pasow": 2,
"klasa": "GP"
},
{
  "id": "N4",
  "type": "Odcinek drogi",
  "properties": {
    "nazwa": "Autostrada A6"
  }
},
{
  "id": "N5",
  "type": "Odcinek drogi",
  "properties": {
    "nazwa": "Droga lokalna Kurów - Przecław"
  }
},
{
  "id": "N6",
  "type": "Rondo",
  "properties": {
    "nazwa": "Rondo Hakena",
    "lokalizacja": "Szczecin"
  }
},
{
  "id": "N7",
  "type": "Rondo",
  "properties": {
    "nazwa": "Rondo południowe",
    "lokalizacja": "styk z DK13 na południe od Przecławia"
  }
},
{
  "id": "N8",
  "type": "Rondo",
  "properties": {
    "nazwa": "Rondo Kołbaskowo-Moczyły"
  }
},
{
  "id": "N9",
  "type": "Rondo",
  "properties": {
    "nazwa": "Rondo Rosówek"
  }
},
{
  "id": "N10",
  "type": "Rondo",
  "properties": {
```

```
"nazwa": "Rondo graniczne",
"lokalizacja": "Rosówek - Kamieniec"
},
{
  "id": "N11",
  "type": "Skrzyżowanie",
  "properties": {
    "nazwa": "Węzeł Przecław",
    "rodzaj": "węzeł drogowy"
  }
},
{
  "id": "N12",
  "type": "Skrzyżowanie",
  "properties": {
    "nazwa": "Węzeł Siadło Górne",
    "rodzaj": "węzeł drogowy"
  }
},
{
  "id": "N13",
  "type": "Skrzyżowanie",
  "properties": {
    "nazwa": "Węzeł Szczecin Zachód",
    "rodzaj": "węzeł autostradowy"
  }
},
{
  "id": "N14",
  "type": "Obiekt inżynierski",
  "properties": {
    "typ_obiektu": "Wiadukt",
    "nazwa": "Wiadukt na węzle Przecław"
  }
},
{
  "id": "N15",
  "type": "Obiekt inżynierski",
  "properties": {
    "typ_obiektu": "Wiadukt",
    "nazwa": "Wiadukt w ciągu drogi Kurów - Przecław"
  }
},
{
  "id": "N16",
  "type": "Obiekt inżynierski",
  "properties": {
    "typ_obiektu": "Przepust",
    "nazwa": "Przepusty na obwodnicy"
  }
},
],
"relationships": [
```

```
{ "source": "N6", "target": "N1", "type": "ŁĄCZY_SIE_Z" },
{ "source": "N1", "target": "N7", "type": "ŁĄCZY_SIE_Z" },
{ "source": "N1", "target": "N11", "type": "ZAWIERA" },
{ "source": "N1", "target": "N16", "type": "ZAWIERA" },
{ "source": "N14", "target": "N1", "type": "PRZECHODZI_NAD" },
{ "source": "N15", "target": "N1", "type": "PRZECHODZI_NAD" },
{ "source": "N12", "target": "N1", "type": "ŁĄCZY_SIE_Z" },
{ "source": "N2", "target": "N12", "type": "ŁĄCZY_SIE_Z" },
{ "source": "N2", "target": "N13", "type": "ŁĄCZY_SIE_Z" },
{ "source": "N13", "target": "N4", "type": "ŁĄCZY_SIE_Z" },
{ "source": "N3", "target": "N13", "type": "ŁĄCZY_SIE_Z" },
{ "source": "N3", "target": "N8", "type": "ZAWIERA" },
{ "source": "N3", "target": "N9", "type": "ZAWIERA" },
{ "source": "N3", "target": "N10", "type": "ZAWIERA" }
]
}
```

ZADANIE: Przeanalizuj poniższy tekst i wygeneruj graf JSON zgodnie z powyższym schematem:

```
"""
```

```
def extract_with_gemini(raw_text):
    print("Gemini analizuje tekst z parametrem (temperature=0.0)...")
    model_id = "gemini-2.5-flash"

    try:
        config = types.GenerateContentConfig(
            temperature=0.0,
            response_mime_type="application/json"
        )

        response = client.models.generate_content(
            model=model_id,
            contents=f"{SYSTEM_PROMPT}\n\nTekst do analizy: {raw_text}",
            config=config
        )

        text_response = response.text
        clean_json = text_response.replace('```json', '').replace('```', '').strip()

        start_idx = clean_json.find('{')
        end_idx = clean_json.rfind('}') + 1
        final_json_str = clean_json[start_idx:end_idx]

        parsed_json = json.loads(final_json_str)

        print("\n" + "=" * 50)
        print("WYGENEROWANY STABILNY JSON:")
        print(json.dumps(parsed_json, indent=4, ensure_ascii=False))
        print("=" * 50 + "\n")

        with open("ostatni_wynik.json", "w", encoding="utf-8") as f:
            json.dump(parsed_json, f, indent=4, ensure_ascii=False)
        print("Wynik zapisanego pliku: ostatni_wynik.json")
```



```

    return parsed_json

except Exception as e:
    print(f"Błąd podczas komunikacji z modelem {model_id}: {e}")
    raise e

def save_to_neo4j(data):
    print("Wysyłanie do Neo4j Aura...")
    driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER,
NEO4J_PASSWORD))
    with driver.session() as session:
        for node in data.get('nodes', []):
            query = f"MERGE (n:`{node['type']}` {{id: $id}}) SET n += $props"
            session.run(query, id=node['id'], props=node['properties'])

        for rel in data.get('relationships', []):
            query = f"MATCH (a {{id: $source}}), (b {{id: $target}}) MERGE
(a)-[:{rel['type']}]>(b)"
            session.run(query, source=rel['source'], target=rel['target'])
    driver.close()
    print("Graf zaktualizowany!")

# --- INTERAKTYWNE MENU ---
if __name__ == "__main__":
    DATA_FOLDER = "data"

    # 1. Automatyczne tworzenie folderu na pliki źródłowe, jeśli nie istnieje
    if not os.path.exists(DATA_FOLDER):
        os.makedirs(DATA_FOLDER)
        print(f"Stworzono folder '{DATA_FOLDER}'. Wrzuć tam pliki tekstowe (.txt) i uruchom
program ponownie.")
        exit()

    # 2. Pobranie i odfiltrowanie tylko plików tekstowych .txt
    pliki = [f for f in os.listdir(DATA_FOLDER) if f.endswith('.txt')]

    if not pliki:
        print(f"Folder '{DATA_FOLDER}' jest pusty. Brak tekstów do wyświetlenia.")
        exit()

    # 3. Wyświetlenie menu użytkownikowi
    print("\n" + "=" * 15 + " MENU WYBORU PLIKÓW " + "=" * 15)
    for i, plik in enumerate(pliki, 1):
        print(f" [{i}] -> {plik}")
    print("=" * 50)

    # 4. Pobranie decyzji użytkownika
    wybor_user = input("Wpisz numery plików do przetworzenia oddzielone spacją (np. 1 2
5): ")

    # 5. Przetwarzanie wprowadzonych danych na indeksy listy

```

```

wybrane_indeksy = []
for element in wybor_user.split():
    if element.isdigit():
        indeks = int(element) - 1
        # Zabezpieczenie przed wpisaniem numeru z kosmosu (np. 99, gdy plików jest 3)
        if 0 <= indeks < len(pliki):
            wybrane_indeksy.append(indeks)

# 6. Sprawdzenie, czy użytkownik wybrał cokolwiek sensownego
if not wybrane_indeksy:
    print("Nie wybrano żadnego poprawnego pliku. Koniec programu.")
    exit()

print(f"\nRozpaczynam sekwencyjne przetwarzanie wybranych plików
({len(wybrane_indeksy)})...")

# 7. Główna pętla wykonawcza dla wskazanych plików
for idx in wybrane_indeksy:
    nazwa_pliku = pliki[idx]
    pelna_sciezka = os.path.join(DATA_FOLDER, nazwa_pliku)

    print("\n" + "#" * 30)
    print(f"PRZETWARZAM PLIK: {nazwa_pliku}")
    print("#" * 30)

    try:
        # Odczyt surowego tekstu
        with open(pelna_sciezka, "r", encoding="utf-8") as f:
            tekst_z_pliku = f.read()

        # Procesowanie i ładowanie do Neo4j
        wynik_json = extract_with_gemini(tekst_z_pliku)
        save_to_neo4j(wynik_json)
        print(f"Sukces dla pliku: {nazwa_pliku}")

    except Exception as e:
        print(f"Problem podczas przetwarzania pliku {nazwa_pliku}: {e}")
        print("Przechodzę do kolejnego wybranego zadania...")

print("\nWszystkie wybrane przez Ciebie pliki zostały pomyślnie obsłużone.")

```